

# Experiment No: 8

## Title:

Implement User Authentication using Keycloak (Docker Deployment) for Online Examination System

## Aim:

To implement secure user authentication using Docker-based Keycloak server and to understand its role in providing identity management and access control in an Online Examination System.

## Capstone Scenario – Online Examination System

In an Online Examination System:

- Students must authenticate before attempting exams.
- Faculty must securely upload question papers.
- Admin must manage users and roles.
- Unauthorized access must be prevented.

To achieve secure authentication, we deploy **Keycloak** using **Docker** for easy setup and isolation.

Keycloak provides:

- Secure login portal
- Role-based access control
- Token-based authentication (JWT)
- Single Sign-On (SSO)
- OAuth 2.0 and OpenID Connect support

This experiment simulates secure login for the Online Examination Capstone Project using containerized deployment.

## Theory:

### • X.800 Security Architecture

Authentication service ensures:

- Identity verification

- Access control
- Accountability

#### • **Model for Network Security**

User → Login → Authentication Server → Token → Exam Application

#### • **Identity and Access Management (IAM)**

IAM systems manage:

- Users
- Roles
- Password policies
- Access permissions

#### • **Docker-Based Deployment**

Docker allows:

- Lightweight container execution
- Platform independence
- Quick setup
- Easy rollback and portability

#### • **Keycloak Architecture Components**

- Realm (security domain)
- Client (exam application)
- Users
- Roles
- Tokens (JWT)
- Identity Providers

#### • **Role in Online Examination System**

Example:

- Student → Attempt exam
- Faculty → Upload questions
- Admin → Manage system

## **Algorithm :**

### **Algorithm 1 – Deploy Keycloak using Docker**

1. Install Docker.
2. Pull latest Keycloak image.
3. Run Keycloak container in development mode.
4. Set admin credentials.
5. Access Admin Console via browser.
6. Create Realm.
7. Create Client.
8. Create Roles.
9. Create Users and assign roles.
10. Test login and verify token.

### **Algorithm 2 – Authentication Flow**

1. User accesses exam application.
2. Application redirects to Keycloak login page.
3. User enters credentials.
4. Keycloak verifies user.
5. Keycloak generates JWT access token.
6. User redirected back to application.
7. Application verifies role from token.
8. Access granted or denied.

## **Step-by-Step Procedure (Kali Linux + Docker)**

### **Step 1: Update Kali Linux**

sudo apt update

sudo apt upgrade -y

## Step 2: Install Docker

```
sudo apt install docker.io -y
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
sudo docker --version
```

(Optional: Allow non-root access)

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

## Step 3: Pull Keycloak Docker Image

```
sudo docker pull quay.io/keycloak/keycloak:latest
```

## Step 4: Run Keycloak Container

```
sudo docker run -d -p 8080:8080 \
```

```
-e KEYCLOAK_ADMIN=admin \
```

```
-e KEYCLOAK_ADMIN_PASSWORD=admin123 \
```

```
quay.io/keycloak/keycloak:latest start-dev
```

Keycloak will start on:

<http://localhost:8080>

## Step 5: Login to Admin Console

1. Open browser → <http://localhost:8080>
2. Login using:
  - Username: admin
  - Password: admin123

## Step 6: Create Realm

1. Click Realm dropdown.
2. Select "Create Realm".

3. Name: OnlineExamRealm.
4. Click Create.

### **Step 7: Create Client (Exam Application)**

1. Go to Clients → Create Client.
2. Client ID: exam-app
3. Client Type: OpenID Connect
4. Root URL: <http://localhost:3000>
5. Save.

### **Step 8: Create Roles**

1. Go to Realm Roles.
2. Create:
  - Student
  - Faculty
  - Admin

### **Step 9: Create Users**

1. Go to Users → Add User.
2. Username: student1
3. Set Password.
4. Assign Role: Student

Repeat for Faculty and Admin.

### **Step 10: Test Authentication**

Open:

<http://localhost:8080/realms/OnlineExamRealm/account>

Login with student credentials.

Expected behavior:

- Successful login
- JWT token generated
- Role displayed in token

### **Step 11: View Access Token**

1. Open browser Developer Tools.
2. Check Application → Local Storage.
3. Copy access token.
4. Decode using:

<https://jwt.io>

You will see:

- Username
- Role
- Expiry
- Issuer

### **Step 12: Stop and Remove Container (Optional)**

`docker ps`

`docker stop <container_id>`

`docker rm <container_id>`

### **Expected Output**

When student logs in:

Login Successful

Access Token Generated

Role: Student

Access Granted to Exam Portal

When unauthorized user tries admin panel:

Access Denied – Role Not Authorized

Docker container status:

| CONTAINER ID | IMAGE           | STATUS         |
|--------------|-----------------|----------------|
| xxxxxxx      | keycloak:latest | Up and Running |

### **Result:**

Thus, secure user authentication was successfully implemented using Docker-based Keycloak deployment in the Online Examination System. The experiment demonstrated containerized IAM deployment, role-based access control, and token-based authentication.

### **Conclusion:**

This experiment helps students:

- Understand containerized deployment using Docker
- Implement secure authentication server
- Configure role-based access control